

# ToolBox-RL: Learning to Generalize Tool Use Across Massive Repositories

Xinyan Shi

Nanjing University of Aeronautics and Astronautics  
Nanjing, China  
xynshi@nuaa.edu.cn

Haodong Liu

Researcher  
Shanghai, China  
liuhaodong2008@gmail.com

Renzhi Wang

Nanjing University of Aeronautics and Astronautics  
Nanjing, China  
rzhwang@nuaa.edu.cn

Piji Li\*

Nanjing University of Aeronautics and Astronautics  
Nanjing, China  
lipiji.pz@gmail.com

## Abstract

Recent advances in enabling Large Language Models (LLMs) to use external tools have significantly extended their functional capabilities beyond internal knowledge. However, most existing approaches rely on retrieval-based mechanisms to select suitable tools from massive repositories, which often struggle to align user queries with tool documentation and exhibit limited generalization. To address these issues, we propose **ToolBox-RL**<sup>1</sup>, a novel reinforcement learning framework that unifies query rewriting, intent understanding, and large-scale tool retrieval into an end-to-end optimization process. ToolBox-RL introduces a query rewriting stage to better capture user intent and ensure semantic alignment with tool descriptions, while reinforcement learning encourages autonomous discovery of generalized tool-use strategies through combined cold-start and policy optimization training. Our experiments demonstrate that ToolBox-RL not only achieves the best tool call accuracy on both white-box and black-box tools but also exhibits strong generalization capabilities on out-of-domain dataset. Ablation studies show that ToolBox-RL can adapt to different retrieval methods and maintains good performance even when incorporating a large number of distractor tools.

## CCS Concepts

• **Computing methodologies** → **Natural language generation**;  
• **Information systems** → *Information retrieval*; *Query reformulation*.

## Keywords

Tool Retrieval and Learning, Large Language Models, Reinforcement Learning, Retrieval-Augmented Generation

### ACM Reference Format:

Xinyan Shi, Renzhi Wang, Haodong Liu, and Piji Li. 2026. ToolBox-RL: Learning to Generalize Tool Use Across Massive Repositories. In *Proceedings*

\*Corresponding author.

<sup>1</sup>The code is available at <https://github.com/S-cavy/ToolBox-RL>.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/https://doi.org/10.1145/3774904.3792538>

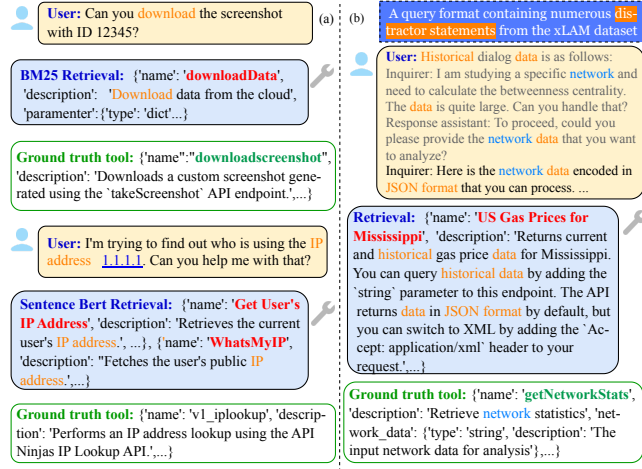
of the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/https://doi.org/10.1145/3774904.3792538>

## 1 Introduction

In recent years, there has been a surge in research focusing on equipping large language models (LLMs) with tool-use capabilities [22]. With external tools, LLMs can acquire capabilities beyond their internal knowledge, such as obtaining up-to-date information through search engines [9], performing complex mathematical computations through programming code interpreters [5, 14], and executing challenging tasks by invoking multiple tools from extensive tool repositories [6].

In tool learning tasks, LLMs encounter numerous complex scenarios. To achieve proper tool invocation, the model must demonstrate robust foundational capabilities, effectively handle complex and diverse inputs—including user intent comprehension, tool selection, and tool document processing—and maintain high generalization across scenarios. However, accuracy and generalization tend to decrease noticeably in more complex settings compared to simpler ones [10]. To enhance LLMs' tool-calling capabilities, existing research typically employs Supervised Fine-Tuning (SFT) [13] or In-Context Learning (ICL) [30] to improve the model's ability to generate tool invocations. Some research introduces Chain-of-Thought (CoT) [7] and reasoning enhancement [26] through the employment of synthetic datasets. Retrieval methods such as MiniLM [25] are also used to allow the model to select appropriate tools from massive external tool API repositories for tool invocation [18, 21]. However, directly applying the BM25 retrieval or sentence embedding models without task-specific training on user queries performs poorly, since there is a misalignment between user queries and tool descriptions [10] as shown in Figure 1(a). Some studies train BERT or MiniLM as a neural tool retriever [20], but training on in-domain tools leads to limited generalization in out-of-domain scenarios and may even result in overfitting. Model performance deteriorates significantly when encountering out-of-domain tools or query descriptions with distractor information, as illustrated in Figure 1(b). Thus, incorporating a stage for the model to understand and rewrite queries in tool learning reasoning steps is necessary.

Recently, reinforcement learning (RL) has shown a significant improvement in LLMs' performance on complex reasoning tasks [15, 16] beyond conventional SFT. It leverages a simple reward scheme



**Figure 1: LLMs and current retrieval method in tool learning may struggle in misalignment existing between user queries and tool descriptions when interacting with massive tool repositories through the retrieval module.**

to encourage the model to autonomously explore better query comprehension and reasoning solutions without direct reasoning supervision, while conventional SFT typically relies on large amounts of high-quality reasoning-supervised data. The effects of RL methodology on tool learning tasks have been validated by recent research such as Tool Integrated Reasoning [3, 4, 12, 24], while previous research has focused exclusively on a small set of tool documents present in the context. Currently, there remains a lack of SFT or RL datasets that simultaneously incorporate query rewriting, user intent comprehension, and access to large-scale tool repositories. LLMs trained on a small tool set with In-Context Learning perform well in their scenarios, but often fail to generalize tool selection beyond known patterns or adaptively decide how to invoke external tools from a vast tool repository since they are not adapted for such tasks [10].

To overcome these issues, we introduce the query rewriting stage and the retrieval module into tool reinforcement learning to enable LLMs to access massive tool repositories. Since reinforcement learning encourages the model to explore autonomously, training can be challenging if the model still lacks the ability to spontaneously obtain rewards during the rollout procedure. Thus, we propose **ToolBox-RL**, a retrieval-oriented two-stage reinforcement learning framework. First, we conduct a cold-start with SFT on a small dataset to allow the model to learn fundamental task capabilities and achieve output format alignment. Second, we apply retrieval-oriented reinforcement learning to further strengthen the model’s reasoning and generalization performance, encouraging the model to think independently and explore the better solution autonomously. Specifically, when the model receives a question or task, it first analyzes the intent and reformulates it in a retrieval-friendly format. The rewritten query is then used for retrieval to obtain the appropriate top-K tool documents. The LLM selects the most suitable tool from the retrieved top-K tool documents to generate the final tool call. We adjust its behavior through feedback

from retrieval and tool-calling results using a compound reward, thereby surpassing the capabilities of SFT alone.

We first construct a high-quality SFT dataset based on xLAM [31] for cold-start. We use both white-box and black-box tools, as well as fully out-of-domain tools from the other dataset, as the test set. White-box tool descriptions may be included in the SFT dataset, while black-box tools are considered out-of-domain relative to the SFT data. ToolBox-RL demonstrates consistent improvements over the baseline across models of various scales. For in-domain tools, ToolBox-7B shows a 5.40% improvement in tool call accuracy over the baseline. For out-of-domain tools, ToolBox-7B attains a 1.80% improvement in tool call accuracy and also exhibits strong generalization capability on the fully out-of-domain dataset. After applying ToolBox-RL on a sufficiently large and high-quality dataset, the performance of LLMs on smaller datasets generalizes more effectively than models trained directly on the same datasets for an equivalent number of epochs. Moreover, we find that reinforcement learning can allow the model to autonomously explore solutions and restore the out-of-domain retrieval capability that may degrade due to SFT, thereby further enhancing generalization.

In addition, we performed detailed ablation studies on the implementation of the retrieval method and tool repository scales, further validating the effectiveness and generalization capability of our training approach. Our approach can adapt to different retrieval methods, including traditional IR techniques such as BM25. Even for tool repositories containing up to 18K tools, our approach maintains superior performance. This indicates that even when black-box tools and distractor tools are added to the tool repository, ToolBox-RL can still achieve strong generalization performance. Our contributions are as follows:

- We propose ToolBox-RL, a two-stage training framework to enhance the tool usage capability of LLMs with retrieval and massive tool repositories, including: (1) Cold-start fine-tuning to ensure that LLMs learn basic format and task abilities; (2) Rule-based reinforcement learning that encourages the model to explore better generalized solutions through a compound reward, promoting effective query rewriting, user intent comprehension and tool call generation.
- We conducted tests on two datasets to verify the effectiveness of the ToolBox-RL method. Through testing with white-box tools, black-box tools and an out-of-domain test set, we demonstrated the strong generalization capability of the ToolBox-RL method on out-of-domain examples.
- We conducted ablation studies across different retrieval methods, as well as different tool repository sizes, to further validate the generalization capability of ToolBox-RL, providing new insights into reinforcement learning for tool usage with access to large-scale tool repositories.

## 2 Related work

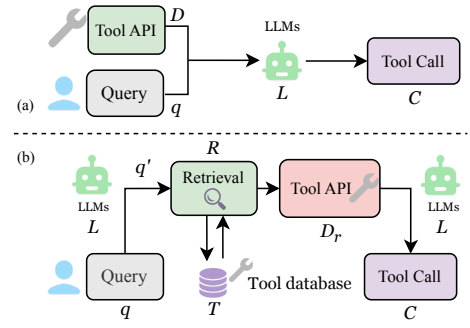
**Tool learning.** Enabling LLMs to use external tools leads to great improvements across various tasks. For tools of varying scales, tool selection methods can be divided into two categories. When the number of candidate tools is manageable, their descriptions can be directly concatenated into the model input and used with templates such as Chain-of-Thought (CoT) or ReACT [27] to guide the model in determining which tool to invoke at this step. When the

number of tools is large, an external retriever is typically employed to filter the top-K potentially relevant tools. Common retrieval methods include traditional Information Retrieval (IR) techniques such as BM25 and semantic vector retrieval such as Sentence-BERT. Some classic tool learning approaches utilize just several tools such as search engines and code interpreters. Meanwhile, research using extensive tool repositories and retrieval module also emerges, such as ToolBench [20] and APIBank [10]. These approaches use a retrieval module to obtain tool documents, addressing the issue of tool repository updates and the limitation of ICL when using massive tool documents. Existing research on improving tool learning capabilities falls mainly into two categories. One focuses on improving LLMs through training, such as using large-scale high-quality datasets for SFT [5] or reinforcement learning methods like DPO [29] and GRPO [19]. SFT approaches rely heavily on training data and generally offer moderate generalization performance across different tasks. The other category is based on prompt and context learning methods [17], where the tool-use capabilities of LLMs are enhanced through prompt engineering [33]. A training-free approach offers ease of use, however, models utilizing ICL often tend to fail when handling complex tasks.

**Reinforcement Learning for reasoning and tool learning.** Recent progress in reinforcement learning (RL) has notably bolstered the reasoning abilities of language models. OpenAI-o1 [8] and QwQ [23] demonstrate the effectiveness of RL in large-scale models, while DeepSeek-R1 [2] shows the effectiveness of reinforcement learning with simple rules in reasoning. For small models, LIMO [28] and LIMR [11] also reflect the effectiveness of RL. Moreover, reasoning-oriented reinforcement learning has been proven effective on other tasks, such as Research [1] on search and reasoning tasks. In the field of tool learning, some research integrates several external tools, especially search engines and code interpreters to improve reasoning, which is named Tool Integrated Reasoning (TIR) [5], with notable examples including ToRL [12], Re-Tool [4], and OTC [24]. Additionally, there has been work using reinforcement learning in general tool learning invocation, such as Nemotron-Research-Tool-N1 [32] and multi-tools case such as Tool-Star [3]. All above tasks provide several tool documents directly within the context, with limitations in tool quantity and tool selection. In contrast, our work aims to enhance the generalization of LLMs' tool-calling capabilities in flexible tool-use scenarios, where tools from massive repositories with complex functionalities and argument structures are available through retrieval.

### 3 Problem Formulation

Since the tool APIs return deterministic results when the parameters are invoked correctly, our focus is on ensuring that the model generates the correct tool invocation. As shown in Figure 2, for tool learning tasks that involve retrieval modules and external tool repositories, when given a query, the objective is to select the appropriate tool from the external tool repository and generate the tool invocation with correct parameters. Specifically, given a task query  $q$  and external tool repositories  $\mathcal{T} = \{t_i\}$  that the LLM  $L$  can access, each tool  $t_i$  consists of document  $D_i(n_i, d_i, k_i)$ ,  $n_i$  refers to the name of the tool,  $d_i$  refers to the description in natural language that elucidates the functionality or intended purpose of the tool,



**Figure 2: Figures (a) and (b) delineate the tool invocation processes under different tool quantities. Figure (a) shows direct contextual integration of limited tool documents for invocation. Figure (b) illustrates usage with extensive repositories.**

and  $k_i$  denotes the specification of the input parameters associated with the tool.

Since tool documentation descriptions encapsulate the overarching functionalities of tools, whereas user queries articulate specific problems, we leverage LLMs to reformulate queries to mitigate the misalignment between the tool documentation and user queries, thereby optimizing for retrieval effectiveness. Figure 2(a) illustrates the process of tool invocation in scenarios with few tools. In such cases, a limited number of tool documents can be directly included in the context input to the model, which then integrates the documents with the query to produce the final tool invocation. Figure 2(b) illustrates the tool invocation process of LLMs for tasks involving extensive tool repositories. The query, processed by the LLM, is first passed to the retrieval system. The system returns the top-K tool documents that best match the description, which are then input into the LLM along with the context to ultimately obtain the tool invocation. Upon receiving the user's query  $q$ , the model  $L$  rewrites it into a retrieval-friendly query  $q' : L(q) = q'$ . Then we send the rewritten query to the retriever  $R$ , which returns  $k$  most relevant tools as the tool documents  $\mathcal{D}_r$ , based on the match between query and tool descriptions, as follows:

$$R(q', \mathcal{D}(d)) = \{\mathcal{D}_r\}$$

In the equation:  $L(q)$  denotes the rewriting process conducted by the model on the user's query  $q$ , resulting in the rewritten query  $q'$ .  $R(q', \mathcal{D}(d))$  represents the retrieval process, returning the top-K best match tool documents.

The model utilizes the tool set  $\mathcal{D}_r$  along with the above context  $c$ , which includes both  $q$  and  $q'$ , to generate the final tool invocation  $C(n, (k, v))$ , where  $(k, v)$  represents the parameter key-value pair, as follows:

$$L(c, \mathcal{D}_r) = C$$

In the equation:  $L(c, \mathcal{D}_r)$  represents the process conducted by the model to generate the final tool call with the context and the retrieved tool documents. The ultimate goal of tool learning is to generate the correct tool call  $C$  to effectively handle the task or query provided by the user.

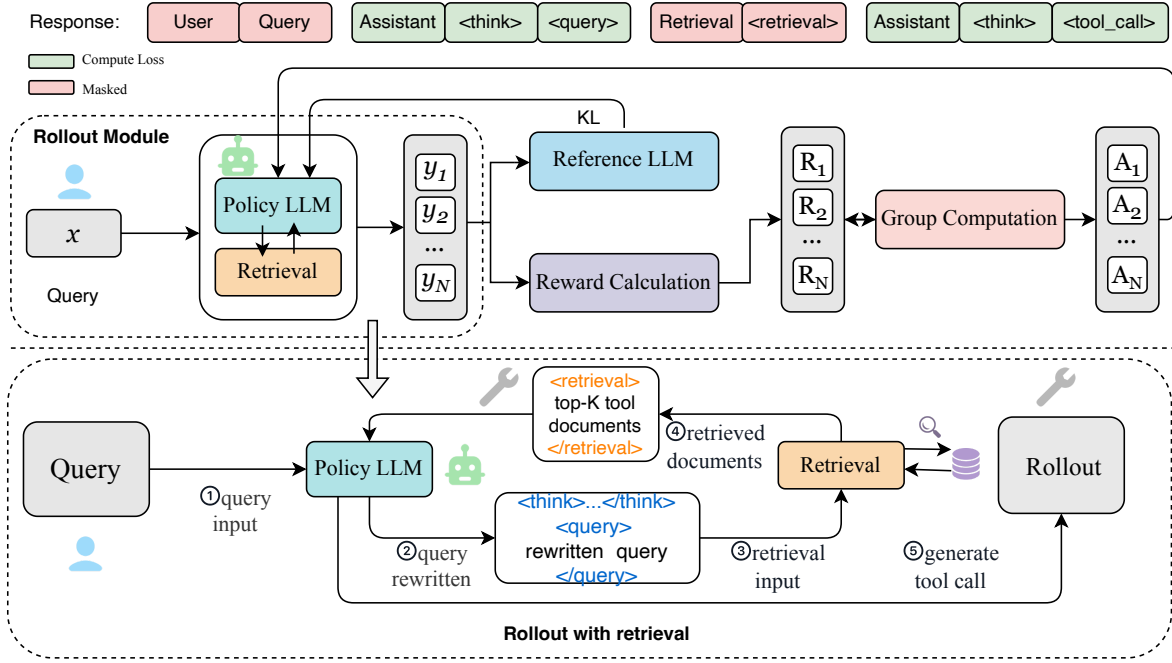


Figure 3: The figure shows schematic of the two-stage reinforcement learning process. The upper part of the figure illustrates the response content and masking scheme, the middle part depicts the overall loss computation and update pipeline, and the lower part demonstrates the detailed rollout procedure with retrieval integration.

## 4 The Proposed ToolBox-RL Framework

Reinforcement learning has demonstrated outstanding performance in training LLMs for multi-turn reasoning, enabling the models to develop reasoning abilities and to exhibit performance beyond the training data.

### 4.1 Rollout for ToolBox-RL

**Thinking Prompt.** To guide the model toward proper reasoning and execution, we design specialized prompts, which are provided in Appendix B. First, when the model receives a user query, we expect it to think about the user’s intent and rewrite the query accordingly with `<think>` and `</think>` tags, enclosing the rewritten query with `<query>` and `</query>` tags. Second, the rewritten query is sent to the retrieval module, then the top-K tool documents returned by the retrieval module are enclosed by `<retrieval>` and `</retrieval>`. Finally, the model generates the tool invocation enclosed within `<tool_call>` and `</tool_call>` based on the retrieved tool documents and contextual information.

**Rollout with retrieval module.** To integrate the retrieval module with model reasoning, we propose a dynamic rollout approach that supports retrieval module. Unlike conventional rollouts, during ToolBox-RL training, our rollout procedure also includes interaction records with the retrieval module. When the generation encounters the `</query>` tag in the model’s response during training, the enclosed content is sent to the retrieval module, and the retrieved top-K tool documents are wrapped with `<retrieval>` and `</retrieval>` tags. Subsequently, the current rollout concatenated with the retrieved documents is used as the next input to iteratively generate

subsequent responses. If an incorrect generation occurs, the model will be instructed to regenerate until a correct tool invocation is produced or the maximum number of dialogue turns is reached.

**Retrieval Result Masking.** We apply Group Relative Policy Optimization (GRPO) as training algorithm, detailed in Section 4.2.2. In the original GRPO method, the loss is computed across all tokens generated within the rollout after the reward calculation. In contrast, the rollouts in ToolBox-RL additionally contain tool documents retrieved from external tool repositories rather than produced by the training policy itself. To avoid introducing bias towards such externally sourced content, we apply masking to the retrieval results when calculating the loss. Concretely, when evaluating the GRPO loss, the loss is restricted to tokens corresponding to text-based reasoning, rewritten queries, and the final tool invocation, while tokens originating from the retrieval module’s documents are excluded from optimization.

### 4.2 The Two-Stage Training

In this section, we propose a two-stage training framework that enhances the LLM’s generation on tool learning tasks with massive tool repositories and retrieval module. It is divided into the cold-start phase and the reinforcement learning phase.

**4.2.1 Cold-Start.** At cold-start stage, we use supervised fine-tuning to calculate the loss. We apply Deepseek-V3 to construct a SFT dataset with chain-of-thought and use target tool description as the rewritten query, with the prompt referenced in Appendix B. Furthermore, to eliminate the interference with the retrieval part caused by multiple tools, we filtered the data, leaving only the data

using a single tool. Each cold-start data instance consists of two rounds of dialogue. In the first round, the model receives a system prompt along with the user’s query. It is required to first reflect on the input, understand the user’s intent, and then generate a rewritten query that is better suited for retrieval. In the second round of dialogue, the rewritten query is fed into the retrieval module, which returns the top-K tool documents. The model is then required to generate the final tool invocation based on both the retrieved documents and the dialogue context.

Cold-start demonstrates significant effectiveness on smaller models, improving their ability to follow prior instructions in long-context scenarios. However, experiments show that while SFT performs well for white-box tools, it does not improve the retrieval accuracy of out-of-domain tools in larger models, such as 7B models, and may even lead to a decrease in generalization. For SFT, utilizing ground truth for model optimization may sometimes result in a decreased success rate in retrieval tasks for tools out of domain. Thus, a second phase of reinforcement learning is required to restore the model’s generalization capability.

**4.2.2 Reinforcement Learning.** During reinforcement learning phase of ToolBox-RL, there is no supervised reasoning data, and we only use a simple reward on rollouts to guide the optimization of LLMs. Our reward function considers the following two parts: answer reward and retrieval reward. The format check is also considered.

**Format check.** We check whether the rollout correctly follows our defined format as described in the prompt templates, mainly checking the correctness of tags. The reward for a given phase can only be computed when the model correctly uses the `<think>` `</think>`, `<query>` `</query>`, or `<tool_call>` `</tool_call>` tags. Otherwise, the corresponding reward is set to zero.

**Answer Reward.** We calculate the correctness of the final tool call between `<tool_call>` and `</tool_call>`, compared with the ground truth via F1 score. Specifically, we compare the JSON key-value pairs and compute the F1 score. In contrast to the next-token prediction approach employed in previous tool learning research, the applied dictionary-based matching offers greater flexibility by allowing variations in argument order without incurring penalties. This mechanism encourages the model to focus on the underlying semantics rather than relying solely on rote memorization.

**Retrieval Reward.** To refine the reward mechanism, we assign rewards for the model’s effective query rewriting. Specifically, for rollouts where the retrieval module returns documents containing the target tool, we consider the query rewriting and retrieval process to be successful and provide a small reward.

For the final reward of a rollout:

$$r = \begin{cases} f1(a_{\text{pred}}, a_{\text{gt}}), & \text{if final answer F1 is not 0} \\ 0.3, & \text{if final answer F1 is 0 but retrieval success} \\ 0, & \text{if final answer F1 is 0 and retrieval fails} \end{cases} \quad (1)$$

where  $a_{\text{pred}}$  is the final answer between `<tool_call>` and `</tool_call>`,  $a_{\text{gt}}$  is the ground truth answer, and  $f1(a_{\text{pred}}, a_{\text{gt}})$  is the F1 score between  $a_{\text{pred}}$  and  $a_{\text{gt}}$ .

**Group Relative Policy Optimization.** We employ Group Relative Policy Optimization (GRPO) as the primary learning algorithm. Distinct from conventional Proximal Policy Optimization (PPO), which necessitates the training of a distinct critic model, GRPO approximates the baseline through a collection of model rollouts to reduce the computational overhead. Multiple rollout chains are sampled to maximize the probability of generating rollouts with higher rewards, as illustrated in Figure 3. Specifically, given the current policy  $\pi_{\theta_{\text{origin}}}$  and a reference policy  $\pi_{\theta_{\text{ref}}}$ , for each input  $x \sim D$ , based on  $N$  rollouts  $\tau = \{y_i\}_{i=1}^N \sim \pi_{\theta_{\text{origin}}}(\cdot|x)$ , the objective of GRPO is to maximize the following function:

$$J(\theta) = \mathbb{E}_{x \sim D, \{y_i\}_{i=1}^N \sim \pi_{\theta_{\text{origin}}}(\cdot|x)} \frac{1}{N} \sum_{i=1}^N \left[ \min \left( \frac{\pi_{\theta}(y_i|x)}{\pi_{\theta_{\text{origin}}}(y_i|x)} \hat{A}_i, \text{clip} \left( \frac{\pi_{\theta}(y_i|x)}{\pi_{\theta_{\text{origin}}}(y_i|x)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_i \right) - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\theta_{\text{ref}}}) \right]$$

Here,

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_j\}_{j=1}^N)}{\text{std}(\{r_j\}_{j=1}^N)}$$

The normalized advantage of the  $i$ -th rollout within the group is denoted as  $\hat{A}_i$ , where  $\epsilon$  signifies the clipping ratio and  $\beta$  denotes the coefficient for the KL divergence loss. Furthermore, the objective function includes a KL divergence penalty to restrict the policy from straying too far from the reference LLM. The GRPO process is depicted in Figure 3.

## 4.3 Additional Details

**Retrieval Method.** The implementation of retrieval in tool learning can be broadly categorized into three types. One type is traditional IR retrieval methods, exemplified by BM25, which is widely used as a ranking function in the field of information retrieval to estimate the relevance between a document  $D$  and a user query  $Q$ . It is an improvement within the probabilistic retrieval framework, and it performs exceptionally well when dealing with long documents and short queries. The second type is vector-based retrieval, in which models such as BERT or MiniLM are employed to compute the cosine similarity between the query and the tool description. The third type involves specific neural network retrievers to enhance search coverage and accuracy, which requires training the retrieval model. However, training the retrieval module may introduce challenges such as overfitting and consume additional computational resources. To ensure generalizability, we chose the first two methods, which are lightweight and do not require additional specialized training.

**White-box Tools and Black-box Tools.** Tools are classified into white-box and black-box categories, which are utilized as one of the criteria for partitioning the test set. We define tools for which the model can access the ground truth description as white-box tools, whereas tools whose ground truth descriptions are not directly available to the model during training are referred to as black-box tools. Black-box tools represent a form of out-of-domain evaluation. In addition, we further consider a fully out-of-domain scenario by training the model on only one dataset and evaluating it on a different dataset.

| Model                              | xLAM-black-box |              |              |              | API-Bank     |              |              |              | xLAM-white-box |              |              |              |
|------------------------------------|----------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|----------------|--------------|--------------|--------------|
|                                    | ACC            | F1           | RA           | MRR          | ACC          | F1           | RA           | MRR          | ACC            | F1           | RA           | MRR          |
| <b>Instruct Model</b>              |                |              |              |              |              |              |              |              |                |              |              |              |
| Qwen-2.5-1.5B                      | 00.00          | 00.00        | 37.10        | 32.47        | 00.00        | 00.00        | 16.11        | 12.69        | 00.00          | 00.00        | 42.90        | 37.70        |
| Qwen-2.5-3B                        | 05.10          | 05.10        | 12.40        | 10.45        | 04.44        | 04.44        | 13.33        | 10.09        | 05.50          | 05.50        | 10.50        | 09.10        |
| Qwen-2.5-7B                        | 17.50          | 17.50        | 67.30        | 57.65        | 09.17        | 09.17        | 44.72        | 36.39        | 25.60          | 25.60        | 70.60        | 61.25        |
| <b>Reasoning or specific Model</b> |                |              |              |              |              |              |              |              |                |              |              |              |
| Deepseek-R1-8B                     | 00.00          | 00.00        | 01.40        | 01.18        | 00.00        | 00.00        | 01.67        | 01.39        | 00.00          | 00.00        | 03.70        | 03.27        |
| Research-Qwen-7B                   | 08.30          | 08.30        | 68.70        | 57.22        | 00.83        | 00.83        | 46.94        | 36.44        | 13.40          | 13.40        | 71.30        | 61.52        |
| Tool-Star-7B                       | 00.00          | 00.00        | 18.70        | 06.23        | 00.00        | 00.00        | 13.89        | 11.81        | 00.00          | 00.00        | 17.10        | 14.95        |
| ToolRL-7B                          | 42.40          | 42.40        | 66.50        | 57.00        | 23.33        | 23.33        | 44.44        | 35.69        | 50.40          | 50.40        | 68.90        | 59.53        |
| xLAM-2-7B-FC-R                     | 23.10          | 23.10        | 45.10        | 37.82        | 24.72        | 24.72        | 46.39        | 37.08        | 34.40          | 34.40        | 54.20        | 46.53        |
| Hammer2.1-7B                       | 00.00          | 00.00        | 57.50        | 49.55        | 00.00        | 00.00        | 37.50        | 30.93        | 00.00          | 00.00        | 52.40        | 44.37        |
| <b>Ours Intermediate Model</b>     |                |              |              |              |              |              |              |              |                |              |              |              |
| Qwen-2.5-1.5B+SFT                  | 09.20          | 09.53        | 56.40        | 48.52        | 03.33        | 03.70        | 27.50        | 23.84        | 09.80          | 09.93        | 66.60        | 58.00        |
| Qwen-2.5-3B+SFT                    | 32.50          | 33.70        | 57.40        | 49.53        | 30.83        | 31.57        | 52.22        | 38.52        | 46.50          | 46.90        | 68.30        | 60.33        |
| Qwen-2.5-7B+SFT                    | 39.10          | 40.35        | 59.80        | 50.80        | 31.94        | 32.31        | 52.78        | 39.12        | 51.10          | 51.75        | 70.30        | 61.82        |
| Qwen-2.5-1.5B+RL                   | 00.00          | 00.00        | 59.20        | 51.98        | 00.00        | 00.00        | 35.28        | 28.33        | 00.00          | 00.00        | 63.80        | 55.58        |
| Qwen-2.5-3B+RL                     | 23.20          | 23.20        | 47.00        | 38.65        | 16.39        | 16.39        | 36.67        | 28.15        | 29.30          | 29.30        | 54.20        | 46.50        |
| Qwen-2.5-7B+RL                     | 44.00          | 44.00        | <b>70.60</b> | 60.13        | 26.67        | 26.67        | 50.28        | 38.84        | 51.40          | 51.40        | 74.70        | 64.58        |
| <b>ToolBox-RL</b>                  |                |              |              |              |              |              |              |              |                |              |              |              |
| <b>ToolBox-1.5B</b>                | 34.90          | 35.57        | 59.30        | 50.62        | 07.22        | 07.59        | 27.78        | 24.44        | 46.60          | 46.93        | 69.70        | 60.95        |
| <b>ToolBox-3B</b>                  | <u>36.50</u>   | <u>37.30</u> | <u>60.00</u> | <u>52.18</u> | <u>30.83</u> | <u>31.39</u> | <u>52.78</u> | <u>38.70</u> | <u>49.10</u>   | <u>49.43</u> | <u>70.40</u> | <u>63.07</u> |
| <b>ToolBox-7B</b>                  | <b>44.20</b>   | <b>45.27</b> | 68.90        | <b>56.77</b> | <b>32.22</b> | <b>32.59</b> | <b>55.00</b> | <b>41.30</b> | <b>56.10</b>   | <b>56.57</b> | <b>75.30</b> | <b>66.32</b> |

**Table 1: The main experimental results are presented in the table, where we highlight the best metrics among 7B to 8B models in bold and the best metrics among 1.5B and 3B models with underline.**

| Model        | ACC                    | F1                     | RA                     | MRR                    |
|--------------|------------------------|------------------------|------------------------|------------------------|
| ToolBox-1.5B | 13.61 <sup>+6.39</sup> | 13.61 <sup>+6.02</sup> | 29.17 <sup>+1.39</sup> | 25.00 <sup>+0.56</sup> |
| ToolBox-3B   | 32.22 <sup>+1.39</sup> | 32.41 <sup>+1.02</sup> | 53.06 <sup>+0.28</sup> | 39.17 <sup>+0.47</sup> |
| ToolBox-7B   | 34.72 <sup>+2.50</sup> | 34.72 <sup>+2.13</sup> | 57.50 <sup>+2.50</sup> | 42.92 <sup>+1.62</sup> |

**Table 2: We tested the models trained solely on xLAM using the API-Bank dataset and the performance of the models increased overall.**

## 5 Experiment

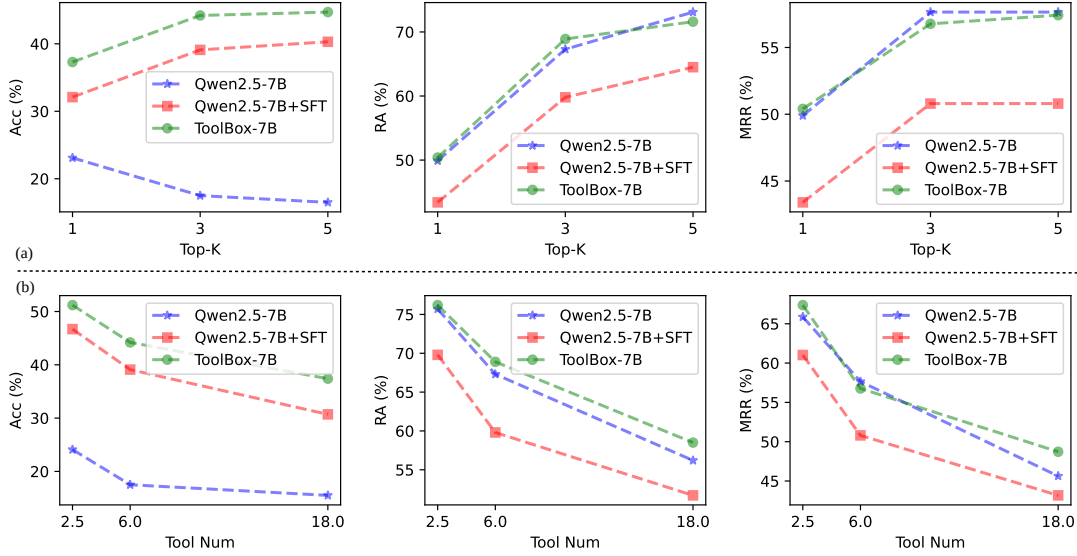
### 5.1 Experiment Setting

**Datasets.** For our task, we choose xLAM [31], a compositional dataset with massive tool documents, as our main training dataset. More details can be found in Appendix A. We filter the multi-tool examples and leave 32K for our experiment. We also test our method on API-Bank, a benchmark specifically designed for tool-augmented LLMs. For cold-start, we sample 5K examples from xLAM and employ DeepSeek-V3 to assist in constructing the chain-of-thought. For RL, we use 32K examples from xLAM and 3K examples from API-Bank respectively. To evaluate the generalization capability of our approach, we constructed two 1K test sets based on xLAM: (1) a black-box tool test set where none of the tools appeared in the SFT training data, and (2) a mixed white-box tool dataset containing tools that were present in the SFT training set. We also use 360

black-box tool examples in API-Bank evaluation. For each dataset respectively, we constructed a tool repository consisting of tools that are involved in examples and other unused distractor tools. To better illustrate the impact of in-domain versus out-of-domain tools, as well as white-box versus black-box tools, our test tool repository and training tool repository are not identical, but both contain the tool documents required for each phase. For example, in the xLAM tool repository, the training tool repository contains 13K tool entities. The test tool repository for xLAM includes 6K tool entities, more than half of which are distractor tools. More details are presented in Appendix D. Additionally, we evaluate our model’s performance by training on xLAM and testing on the API-Bank benchmark, as a fully out-of-domain case.

**Retrieval module.** We employ two methods for retrieval: MiniLM-v6 and BM25. The former is used primarily in our experiment since vector-based retrieval is more popular for tool learning tasks, while ablation experiment for BM25 are detailed in the section. We use MiniLM-v6 to compute semantic similarity via cosine similarity. The retrieval module will return the top-K matched documents fetched from the tool repository.

**Metrics.** For the tool call evaluation, we assess based on two metrics: Accuracy (Acc) and F1 score. Accuracy measures the probability of generating completely correct tool calls, while F1 score evaluates the parameter accuracy and recall when the tool name



**Figure 4: Figure (a) shows the performance on black-box xLAM test set under different top-K parameters, while Figure (b) illustrates the performance on black-box xLAM test set with varying numbers of tools in the tool repository.**

is correctly identified. Additionally, for the retrieval step, we introduce Retrieval Accuracy (RA), which quantifies the proportion of successful retrieval. When retrieval result contains the target tool document, we consider the rewriting and retrieval are success. We also incorporate Mean Reciprocal Rank (MRR) to measure the ranking quality of retrieved results.

**Implementation Details.** For cold-start, we use LoRA for 3 epochs on the 5K cold-start dataset, training settings detailed in Appendix E. For RL, we conduct all RL experiments using the verL<sup>2</sup> framework and adopt the GRPO algorithm detailed in Section 4.2.2. For each training step, we sample a batch of 256 samples, and generate 8 responses for per query, training for 3 epochs in total (see Appendix E for more configuration details). We set maximum dialogue-turn limit as 3, and if the model makes mistake, it will receive the mistake prompt and try to generate a new answer, until the dialogue turn up to maximum. We initialize our models with the Qwen2.5-1.5B-Instruct, Qwen2.5-3B-Instruct and Qwen2.5-7B-Instruct which are further tuned under the GRPO objective with our customized reward design. The retrieval top-K is set to 3. For evaluation, we set max tokens to 4096 to contain the long context including tool documents. We set the evaluation temperature as 0.2 and sample 5 times.

**Baselines.** We selected three models as baselines to complete the retrieval task. (1) Basic Instruct Model: The original model without any additional fine-tuning or reinforcement learning, evaluated using the same prompts. (2) Reasoning or specific model: Models trained with reinforcement learning on reasoning tasks, such as DeepSeek-V3-8B for comprehensive tasks and Research-7B for TIR. Tool learning models trained with reinforcement learning on a limited set of tool documents, such as ToolRL-7B (trained with RL on xLAM data) and Tool-Star-7B (multi-tool RL method). Tool

models trained with supervised fine-tuning, such as hammer-7B and xLAM-7B (trained on xLAM data). (3) Ours Intermediate Model: The models are categorized into two groups, those trained exclusively with cold-start supervised fine-tuning (SFT), and those trained directly with reinforcement learning (RL).

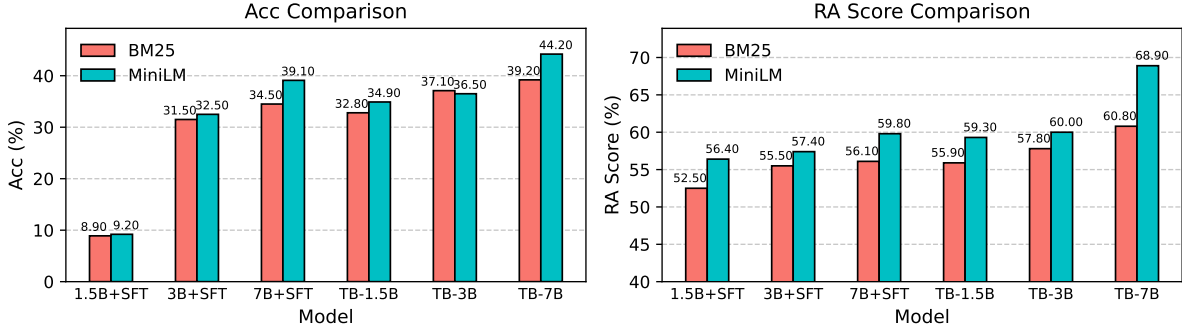
## 5.2 Main result

We conducted experiments on 3 basic models and 6 baselines. Our training method resulted in significant improvements for the task. As shown in Table 1, specifically, ToolBox-7B achieved a tool call accuracy of 44.20% and a tool retrieval accuracy of 68.90% on black-box tools in the xLAM, exceeding the ToolRL baseline (also trained on xLAM) by 1.80% and 2.4% respectively. Our black-box tool retrieval performance even surpasses that of specialized search models by 0.20%. On the API-Bank dataset, it achieved a tool call accuracy of 32.22% and a tool retrieval accuracy of 55.00%, surpassing the baseline by 8.89% and 10.56%. On mixed white-box tools, it reached a tool call accuracy of 56.10% and a tool retrieval accuracy of 75.30%, exceeding the baseline by 5.70% and 6.4% respectively.

For baselines, we observe that reasoning models occasionally engage in contemplation without producing appropriate actions. Models with zero accuracy struggle to adhere to the correct output format, failing to generate tool outputs with both accurate tags and content. Furthermore, for longer tool contexts, many models fail to adapt properly, resulting in successful query execution but subsequent failures in tool selection, tool-calling format and tool-call generation.

Tests on out-of-domain test sets indicate that ToolBox-RL not only performs well in-domain but also exhibits good generalization capabilities out-of-domain, achieving optimal results across different datasets. Additionally, we benchmark the training methods, dividing them into solely SFT and solely RL approaches. We found that the cold-start ToolBox-RL training method is superior overall.

<sup>2</sup><https://github.com/volcengine/verl>



**Figure 5: This figure compares the performance on the black-box xLAM tool dataset using the BM25 as retrieval method. We use TB to represent ToolBox.**

High-quality SFT significantly enhances model performance but may reduce generalization in retrieval tasks because the model might fail to learn patterns beyond the data distribution, leading to simple memorization. In contrast, reinforcement learning encourages the model to autonomously explore and discover more effective problem-solving strategies. However, in small models, reinforcement learning can only improve retrieval performance via rewards when the base model struggles to autonomously explore the correct results due to insufficient internal knowledge. Cold-start is particularly effective for this case. For the 7B model, the success rate of retrieval during the intermediate RL stage is highest, as SFT significantly reduces retrieval effectiveness, followed by reinforcement learning which gradually restores and enhances the model’s retrieval capability.

As illustrated in Table 2, we tested the models trained solely on xLAM using the API-Bank dataset. Since xLAM’s data contains 32K examples, the models trained on the xLAM dataset perform better than those trained on 3K examples from API-Bank for the same number of epochs, further demonstrating the generalization capability of ToolBox-RL across the different datasets.

## 6 Ablation Study

### 6.1 Top-K of Retrieval

The number of top-K results returned by retrieval also affects performance. We evaluated three settings with return sizes of 1, 3, and 5 on the black-box xLAM test set. As shown in Figure 4(a), the accuracy on Qwen2.5-7B-Instruct decreases with a larger  $k$ , suggesting that the base model’s performance deteriorates when more tool documentation is introduced in the context, indicating its inability to handle longer retrieved tool documents. In contrast, the trained model demonstrates strong adaptability to longer tool document returns.

### 6.2 Tool Number in the Tool Repository

To investigate the impact of the number of tools in the repository, we constructed three datasets of different tool repository scales, 2.5K, 6K and 18K, based on xLAM and tools in ToolACE [13]. Each dataset contains all tools required by the 1K-example black-box test set, while the remaining tools serve as distractors. As illustrated in Figure 4(b), increasing the number of tools introduces more distractors, which in turn leads to performance degradation across

all metrics. However, our proposed ToolBox-RL method consistently outperforms the baselines. Furthermore, when additional tools are incorporated into the dataset, the relative impact of distractors on ToolBox-RL is smaller, which further substantiates the robustness and generalization ability of our approach on large tool repositories.

### 6.3 Retrieval Method

We conducted experiments on two representative retrieval methods, namely the traditional BM25 and the vector-based MiniLM-v6 in Figure 5. We observed that the model consistently performs better with MiniLM than with BM25, while still maintaining strong performance under the simple retrieval method. This further demonstrates the generalization capability of our approach, showing its adaptability to diverse retrieval methods. Ablation studies demonstrate that ToolBox-RL exhibits strong generalization capability, maintaining robust performance even when faced with black-box tool libraries, the addition of new tools, or changes in retrieval methods and parameters.

## 7 Conclusion

In this work, we introduced ToolBox-RL, a novel two-stage reinforcement learning framework that integrates query rewriting and the retrieval module for large-scale tool repositories directly into the learning process. Experiment results demonstrate that ToolBox-RL effectively mitigates the generalization problems induced by SFT through reinforcement learning. Our approach achieves state-of-the-art tool-call accuracy on both white-box and black-box tools, and exhibits strong cross-dataset generalization even when trained on a single dataset. Ablation studies further confirm the robustness of the ToolBox-RL framework and therefore provide a practical and scalable solution for enhancing LLMs’ tool-use capabilities in complex, dynamic environments.

## Acknowledgments

This research is supported by the National Natural Science Foundation of China (No.62476127), the Natural Science Foundation of Jiangsu Province (No.BK20242039), the Basic Research Program of the Bureau of Science and Technology (ILF24001), the Scientific Research Starting Foundation of Nanjing University of Aeronautics and Astronautics (No.YQR21022), and the High Performance Computing Platform of Nanjing University of Aeronautics and Astronautics.

## References

- [1] Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z. Pan, Wen Zhang, Huajun Chen, Fan Yang, Zenan Zhou, and Weipeng Chen. 2025. ReSearch: Learning to Reason with Search for LLMs via Reinforcement Learning. *CoRR abs/2503.19470* (2025). arXiv:2503.19470 doi:10.48550/ARXIV.2503.19470
- [2] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiusi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuan Chen, Shaoqing Wu, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shutong Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, and Wangding Zeng. 2024. DeepSeek-V3 Technical Report. *CoRR abs/2412.19437* (2024). arXiv:2412.19437 doi:10.48550/ARXIV.2412.19437
- [3] Guanting Dong, Yifei Chen, Xiaoxi Li, Jiajie Jin, Hongjin Qian, Yutao Zhu, Hangyu Mao, Guorui Zhou, Zhicheng Dou, and Ji-Rong Wen. 2025. Tool-Star: Empowering LLM-Brained Multi-Tool Reasoner via Reinforcement Learning. *CoRR abs/2505.16410* (2025). arXiv:2505.16410 doi:10.48550/ARXIV.2505.16410
- [4] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. 2025. ReTool: Reinforcement Learning for Strategic Tool Use in LLMs. *CoRR abs/2504.11536* (2025). arXiv:2504.11536 doi:10.48550/ARXIV.2504.11536
- [5] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujia Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2024. ToG: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*. OpenReview.net. <https://openreview.net/forum?id=EpTtjVoap>
- [6] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *CoRR abs/2503.23278* (2025). arXiv:2503.23278 doi:10.48550/ARXIV.2503.23278
- [7] Tatsuhiro Inaba, Hirokazu Kiyomaru, Fei Cheng, and Sadao Kurohashi. 2023. MultiTool-CoT: GPT-3 Can Use Multiple External Tools with Chain of Thought Prompting. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), ACL 2023, Toronto, Canada, July 9–14, 2023*, Anna Rogers, Jordan L. Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, 1522–1532. doi:10.18653/V1/2023.ACL-SHORT.130
- [8] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, Alex Iftimie, Alex Karpenko, Alex Tachard Passos, Alexander Neitz, Alexander Prokofiev, Alexander Wei, Allison Tam, Ally Bennett, Ananya Kumar, Andre Saraiva, Andrea Vallone, Andrew Duberstein, Andrew Kondrich, Andrey Mishchenko, Andy Applebaum, Angela Jiang, Ashvin Nair, Barret Zoph, Behrooz Ghorbani, Ben Rossen, Benjamin Sokolowsky, Boaz Barak, Bob McGrew, Borys Minaiev, Botao Hao, Bowen Baker, Brandon Houghton, Brandon McKinzie, Brydon Eastman, Camillo Lugaresi, Cary Bassin, Cary Hudson, Chak Ming Li, Charles de Bourcy, Chelsea Voss, Chen Shen, Chong Zhang, Chris Koch, Chris Orsinger, Christopher Hesse, Claudia Fischer, Clive Chan, Dan Roberts, Daniel Kappler, Daniel Levy, Daniel Selsam, David Dohan, David Farhi, David Mely, David Robinson, Dimitris Tsipras, Doug Li, Dragos Oprica, Eben Freeman, Eddie Zhang, Edmund Wong, Elizabeth Proehl, Enoch Cheung, Eric Mitchell, Erik Wallace, Erik Ritter, Evan Mays, Fan Wang, Felipe Petroski Such, Filippo Raso, Florencia Leoni, Foivos Tsimpourlas, Francis Song, Fred von Lohmann, Freddie Sulit, Geoff Salmon, Giambattista Parascandolo, Gildas Chabot, Grace Zhao, Greg Brockman, Guillaume Leclerc, Hadi Salman, Haiming Bao, Hao Sheng, Hart Andrin, Hessam Bagherinezhad, Hongyu Ren, Hunter Lightman, Hyung Won Chung, Ian Kivlichen, Ian O’Connell, Ian Osband, Ignasi Clavera Gilaberte, and Ilge Akkaya. 2024. OpenAI o1 System Card. *CoRR abs/2412.16720* (2024). arXiv:2412.16720 doi:10.48550/ARXIV.2412.16720
- [9] Bowen Jin, Hansi Zeng, Zhenrui Yue, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. *CoRR abs/2503.09516* (2025). arXiv:2503.09516 doi:10.48550/ARXIV.2503.09516
- [10] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6–10, 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, 3102–3116. doi:10.18653/V1/2023.EMNLP-MAIN.187
- [11] Xuefeng Li, Haoyang Zou, and Pengfei Liu. 2025. LLMR: Less is More for RL Scaling. *CoRR abs/2502.11886* (2025). arXiv:2502.11886 doi:10.48550/ARXIV.2502.11886
- [12] Xuefeng Li, Haoyang Zou, and Pengfei Liu. 2025. ToRL: Scaling Tool-Integrated RL. *CoRR abs/2503.23383* (2025). arXiv:2503.23383 doi:10.48550/ARXIV.2503.23383
- [13] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yunnan Yu, Zezhong Wang, Yuxian Wang, Wu Ning, Yutai Hou, Bin Wang, Chuhan Wu, Xinzhi Wang, Yong Liu, Yasheng Wang, Duyu Tang, Dandan Tu, Lifeng Shang, Xin Jiang, Ruiming Tang, Defu Lian, Qun Liu, and Enhong Chen. 2024. ToolACE: Winning the Points of LLM Function Calling. *CoRR abs/2409.00920* (2024). arXiv:2409.00920 doi:10.48550/ARXIV.2409.00920
- [14] Yifei Lu, Fanghua Ye, Jian Li, Qiang Gao, Cheng Liu, Haibo Luo, Nan Du, Xiaolong Li, and Feiliang Ren. 2025. CodeTool: Enhancing Programmatic Tool Invocation of LLMs via Process Supervision. *CoRR abs/2503.20840* (2025). arXiv:2503.20840 doi:10.48550/ARXIV.2503.20840
- [15] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Guanjin Xiong, and Hongsheng Li. 2025. UI-R1: Enhancing Action Prediction of GUI Agents by Reinforcement Learning. *CoRR abs/2503.21620* (2025). arXiv:2503.21620 doi:10.48550/ARXIV.2503.21620
- [16] Peixian Ma, Xialie Zhuang, Chengjin Xu, Xuhui Jiang, Ran Chen, and Jian Guo. 2025. SQL-R1: Training Natural Language to SQL Reasoning Model By Reinforcement Learning. *CoRR abs/2504.08600* (2025). arXiv:2504.08600 doi:10.48550/ARXIV.2504.08600
- [17] Bhargavi Paranjape, Scott M. Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. 2023. ART: Automatic multi-step reasoning and tool-use for large language models. *CoRR abs/2303.09014* (2023). arXiv:2303.09014 doi:10.48550/ARXIV.2303.09014
- [18] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large Language Model Connected with Massive APIs. *CoRR abs/2305.15334* (2023). arXiv:2305.15334 doi:10.48550/ARXIV.2305.15334
- [19] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. 2025. ToolRL: Reward is All Tool Learning Needs. *CoRR abs/2504.13958* (2025). arXiv:2504.13958 doi:10.48550/ARXIV.2504.13958
- [20] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*. OpenReview.net. <https://openreview.net/forum?id=dHng2O0jir>
- [21] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *CoRR abs/2307.16789* (2023). arXiv:2307.16789 doi:10.48550/ARXIV.2307.16789
- [22] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. 2024. Tool Learning with Large Language Models: A Survey. *CoRR abs/2405.17935* (2024). arXiv:2405.17935 doi:10.48550/ARXIV.2405.17935
- [23] Qwen Team. 2024. QwQ: Reflect deeply on the boundaries of the unknown. *Hugging face* (2024).
- [24] Hongru Wang, Cheng Qian, Wanjun Zhong, Xiusi Chen, Jiahao Qiu, Shijue Huang, Bowen Jin, Mengdi Wang, Kam-Fai Wong, and Heng Ji. 2025. OTC: Optimal Tool Calls via Reinforcement Learning. *CoRR abs/2504.14870* (2025). arXiv:2504.14870 doi:10.48550/ARXIV.2504.14870
- [25] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6–12, 2020, virtual*, Hugo Larochelle, Marc Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.). <https://proceedings.neurips.cc/paper/2020/hash/3f5ee243547dee91fdb053c1c4a845aa-Abstract.html>
- [26] Shirley Wu, Shiyu Zhao, Qian Huang, Kexin Huang, Michihiro Yasunaga, Kaidi Cao, Vassilis N. Ioannidis, Karthik Subbian, Jure Leskovec, and James Y. Zou. 2024. AvaTaR: Optimizing LLM Agents for Tool Usage via Contrastive Reasoning. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 – 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). [http://papers.nips.cc/paper\\_files/paper/2024/hash/2db8ce969b000fe0b3fb172490c33ce8-Abstract-Conference.html](http://papers.nips.cc/paper_files/paper/2024/hash/2db8ce969b000fe0b3fb172490c33ce8-Abstract-Conference.html)
- [27] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1–5, 2023*. OpenReview.net. [https://openreview.net/forum?id=WE\\_vluYUL-X](https://openreview.net/forum?id=WE_vluYUL-X)

- [28] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. 2025. LIMO: Less is More for Reasoning. *CoRR* abs/2502.03387 (2025). arXiv:2502.03387 doi:10.48550/ARXIV.2502.03387
- [29] Yuanqing Yu, Zhefan Wang, Weizhi Ma, Zhicheng Guo, Jingtao Zhan, Shuai Wang, Chuhan Wu, Zhiqiang Guo, and Min Zhang. 2024. StepTool: A Step-grained Reinforcement Learning Framework for Tool Learning in LLMs. *CoRR* abs/2410.07745 (2024). arXiv:2410.07745 doi:10.48550/ARXIV.2410.07745
- [30] Xingshan Zeng, Weiwen Liu, Xu Huang, Zezhong Wang, Lingzhi Wang, Liangyou Li, Yasheng Wang, Lifeng Shang, Xin Jiang, Ruiming Tang, and Qun Liu. 2025. ToolACE-R: Tool Learning with Adaptive Self-Refinement. *CoRR* abs/2504.01400 (2025). arXiv:2504.01400 doi:10.48550/ARXIV.2504.01400
- [31] Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, Weiran Yao, Juntao Tan, Akshara Prabhakar, Haolin Chen, Zhiwei Liu, Yihao Feng, Tulika Manoj Awalgaoonkar, Rithesh R. N., Zeyuan Chen, Ran Xu, Juan Carlos Niebles, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. 2025. xLAM: A Family of Large Action Models to Empower AI Agent Systems. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, 11583–11597. doi:10.18653/V1/2025.NAACL-LONG.578
- [32] Shaokun Zhang, Yi Dong, Jieyu Zhang, Jan Kautz, Bryan Catanzaro, Andrew Tao, Qingyun Wu, Zhiding Yu, and Guilin Liu. 2025. Nemotron-Research-Tool-N1: Exploring Tool-Using Language Models with Reinforced Reasoning. *CoRR* abs/2505.00024 (2025). arXiv:2505.00024 doi:10.48550/ARXIV.2505.00024
- [33] Shaokun Zhang, Jieyu Zhang, Dujian Ding, Mirian Hipolito Garcia, Ankur Mallick, Daniel Madrigal, Menglin Xia, Victor Rühle, Qingyun Wu, and Chi Wang. 2024. EcoAct: Economic Agent Determines When to Register What Action. *CoRR* abs/2411.01643 (2024). arXiv:2411.01643 doi:10.48550/ARXIV.2411.01643

## A Datasets

The xLAM dataset contains massive APIs and high quality examples including single tool calls and multi tool calls. It leverages over 3,673 real-world APIs from ToolBench across 21 categories to generate 60,000 high-quality data samples, utilizing a systematic data synthesis framework which can generate verifiable datasets based on a collection of executable APIs. Each data example contains the user query, the set of tools available to the LLM, and the correct API call. In our work, we extract the corresponding tool documentation and incorporate it into the tool repository. For larger tool repository and ablation study on tool repository scale, we also add tool APIs of ToolACE [13], which has a comprehensive API pool of 26,507 APIs, to the xLAM tool repositories as distractor tools.

**Table 3: Cold-start SFT dataset Construct Prompt for Deepseek-V3.**

| Prompt for thinking before rewriting                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| You need to briefly think with the goal of rewriting the user query to make it closer to a summarized tool description. You should consider the user’s intent and infer the required tool description, then describe your thought process. Please just give out thought process without other things. For example:<br>Case 1: User query: "Please get me the top health news in German from Germany, and also the latest news from the BBC source." Target tool description: "Fetches the latest news from Google News for a specified language region."<br>Case 2: User query: "I'm trying to find all the movies directed by Steven Spielberg and the ones starring Tom Hanks. Can you also get me the episodes of the first season of Breaking Bad? And while you're at it, fetch me a list of 5 random actors." Tool description: "Fetches movies associated with a given cast member's name from the IMDB API." Now, rewrite and analyze the following user query: |

### Prompt for Rollouts

Answer the given question with proper tool calling. You must conduct reason-ing inside `<think>` and `</think>` first every time you get new information. Then you need to rewrite the query user given to a better new query by `<query>` new query `</query>` and it will return the top searched tool information between `<retrieval>` and `</retrieval>`. You need to briefly think with the goal of rewriting the user query to make it closer to a summarized tool description. You should consider the user's intent and infer the required tool description. You can provide the final tool calling inside `<tool_call>` and `</tool_call>`, without de-tailed illustrations. For example, `<tool_call> {"name": "getplatformlist", "para-meters": { "page": "1" }}` `</tool_call>`. Please remember this platform and strictly follow it.  
Now the question is as follows:

**Figure 6: System prompt for ToolBox-RL rollout in training and inference in evaluation.**

## B Prompt

Prompt for thinking before rewriting shows in Table 3 and prompt for rollouts is illustrated in Figure 6.

**Table 4: Implementation details of SFT.**

| Parameter                 | Value |
|---------------------------|-------|
| LoRA Rank                 | 8     |
| LoRA Target               | all   |
| Number of Training Epochs | 3     |
| Learning Rate             | 1e-4  |
| Warmup Ratio              | 0.1   |

**Table 5: Implementation details of RL.**

| Parameter                 | Value |
|---------------------------|-------|
| Learning Rate             | 1e-6  |
| Train Batch Size          | 256   |
| Number of Training Epochs | 3     |
| Number of Rollouts        | 8     |
| Rollout Temperature       | 1.0   |
| KL Loss Coefficient       | 0.001 |

## C Metrics

For an evaluation set containing  $Q$  queries, MRR is calculated as:  $MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$  where  $|Q|$  represents Total number of queries,  $rank_i$  represents rank position (starting from 1) of the first relevant result for the  $i$ -th query, if no relevant result exists for a query,  $\frac{1}{rank_i} = 0$ .

## D Tool Repositories

For RL training on xLAM, we use 32K examples, and the tool repository contains 13K tool documents. For the xLAM evaluation, we use 1K test examples for both black-box and white-box tools, and the tool repository contains 6K tool documents. For RL training on API-Bank, we use 3K examples, and the repository contains 3K tool documents. For the API-Bank evaluation, we use 360 test examples

for models trained on API-Bank and models trained only on xLAM, and the tool repository contains 360 tool documents.

## E Training Details

Our SFT training is conducted on 1 Nvidia A100 GPU with LoRA. We show the hyperparameter settings in Table 4. Our RL training is conducted on 8 Nvidia A100 GPUs and we show the hyperparameter settings in Table 5.